ESPECIFICAÇÃO DE ENGENHARIA DE DADOS & ARQUITETURA DE SOFTWARE

ORIENTAÇÃO TÉCNICA DE PROGRAMAÇÃO: PADRÃO CANÔNICO DA MATRIZ DE PRINCÍPIOS ATIVOS E VÍNCULO RELACIONAL COM A BASE MESTRE

Contrato Canônico de 13 Campos Químico-Fiscais, Relacionamento Relacional 1:N com o Cadastro Mestre de Produtos e Pipeline Transacional de Ingestão Atômica (Dual-Sync)

Este documento técnico detalha as diretrizes de modelagem relacional, esquemas de validação (Pydantic v2 / SQLAlchemy) e pipelines de integração para que o **Cadastro de Princípios Ativos e Composições** opere com **estrita conformidade aos 13 campos** da aba `4. Matriz Principios Ativos` da planilha mestre consolidada (`base_oficial_consolidada_produtos_agro.xlsx`). Estabelece a amarração relacional de integridade com o **Cadastro Mestre de Produtos (28 campos)**, garantindo que toda nova integração a portais oficiais (AGROFIT, SIPEAGRO e DOU) alimente simultânea e atomicamente ambas as tabelas sem redundâncias ou divergências tributárias.

Artefato Referência	`base_oficial_consolidada_produtos_agro.xlsx` (Aba 4: Matriz Princípios Ativos)
Tabelas Relacionadas	`matriz_principios_ativos` (13 campos) ↔ `cadastro_mestre_produtos` (28 campos)
Cardinalidade Relacional	1:N (1 Princípio Ativo / Composição governa *N* Produtos Comerciais e Técnicos)
Mecanismo de Ingestão	Pipeline Transacional Atômico (Dual-Sync Ingestion Pipeline com Rollback ACID)
Stack Tecnológica	Python 3.11+, Pydantic v2, SQLAlchemy 2.0, FastAPI, PostgreSQL / SQLite
Versão da Especificação	2.5.0 — Homologada para Sistemas ERP e Motores Fiscais do Agronegócio

1. Arquitetura de Relacionamento e Princípios de Domínio

1.1. O Papel Estratégico da Matriz de Princípios Ativos

No ecossistema de insumos agropecuários, a marca comercial é volátil e sujeita a estratégias mercadológicas (clones comerciais, mudanças de titularidade e diferentes embalagens para o mesmo produto). Em contrapartida, a **natureza químico-agronômica** do insumo — consubstanciada no seu princípio ativo, concentração e garantias analíticas — é o elemento basilar que determina a classificação aduaneira na Nomenclatura Comum do Mercosul (NCM/TIPI) e o enquadramento tributário nas alíquotas de PIS/COFINS e futura CBS/IBS.

A **Matriz de Princípios Ativos** desempenha o papel de *Rule Engine* (Motor de Regras Centralizado) do sistema. Ela não armazena produtos comerciais nem embalagens específicas, mas sim as **regras canônicas universais** de cada molécula, mistura ou garantia oficial.

1.2. Modelagem Relacional 1 para N (1:N)

O vínculo relacional entre a Matriz de Princípios Ativos e o Cadastro Mestre de Produtos obedece à cardinalidade **1:N (Um para Muitos)**:

- **1 Registro na Matriz de Princípios Ativos** (ex.: `2,4-D (SAL DIMETILAMINA) 806 G/L SL`) define a NCM padrão (`3808.93.22`), a incidência ou não da Nota de Subposições 2 do Cap. 38, as regras de NCM para as 4 faixas de embalagem e a alíquota de PIS/COFINS (Lei nº 10.925/2004 e LC nº 224/2025).
- **N Produtos Comerciais na Base Mestre** (ex.: `2-4 D AGCN 1X20L`, `AGCN 2,4-D 4X5L`, `MIRANT 1X20L`, `DMA 806 BR 1X20L`, etc.) possuem registros MAPA próprios, titulares distintos e volumes comerciais específicos, mas herdam e validam seus parâmetros aduaneiros e tributários diretamente da respectiva chave da Matriz.

> **Vantagem Arquitetural Crítica:** Caso a Receita Federal publique uma nova Solução de Consulta COSIT alterando a interpretação fiscal de uma molécula (como ocorreu na SC COSIT nº 98.042/2022 para inseticidas da Nota 2 ou na SC COSIT nº 261/2023 para biológicos solubilizadores), basta atualizar **um único registro** na Matriz de Princípios Ativos para que todos os produtos comerciais vinculados na Base Mestre passem automaticamente a refletir o novo enquadramento tributário.

---

2. Dicionário Canônico: Os 13 Campos da Matriz de Princípios Ativos

A tabela a seguir especifica os 13 campos canônicos obrigatórios da Matriz de Princípios Ativos, rigorosamente alinhados à aba `4. Matriz Principios Ativos` da planilha consolidada:

#	CAMPO NA PLANILHA	IDENTIFICADOR SNAKE_CASE	TIPO DE DADO	RESTRIÇÃO / VALIDAÇÃO	EXEMPLO REAL NA BASE
1	**Princípio Ativo / Composição / Garantia**	`principio_ativo_chave`	String(255)	Unique, Not Null, Uppercase Index	`LAMBDA-CIALOTRINA 250 G/L CS`

#	CAMPO NA PLANILHA	IDENTIFICADOR SNAKE_CASE	TIPO DE DADO	RESTRIÇÃO / VALIDAÇÃO	EXEMPLO REAL NA BASE
2	**Macro-Grupo e Sub-Grupo Padrão**	`macro_grupo_subgrupo_padrao`	String(150)	Grupo e Sub-grupo agronômico padrão	`Defensivos — Inseticida / Acaricida`
3	**Nota Subpos. 2 Cap. 38?**	`nota_subpos_2`	Enum	Taxativo: `SIM` ou `NÃO`	`SIM` (Lambda-Cialotrina, Bifentrina)
4	**NCM Padrão (> 10kg/L)**	`ncm_padrao_referencia`	String(10)	Máscara `0000.00.00` (Volume padrão)	`3808.69.90` (Nota 2) ou `3808.93.22`
5	**NCM Faixa 1 (≤ 300g/mL)**	`ncm_faixa1_ate_300g`	String(10)	Máscara `0000.00.00` (≤ 300g/mL)	`3808.61.00` (se Nota 2) ou padrão
6	**NCM Faixa 2 (300g a 7,5kg/L)**	`ncm_faixa2_300g_a_7_5kg`	String(10)	Máscara `0000.00.00` (> 300g a 7,5kg/L)	`3808.62.90` (se Nota 2) ou padrão
7	**NCM Faixa 3 (7,5kg a 10kg/L)**	`ncm_faixa3_7_5kg_a_10kg`	String(10)	Máscara `0000.00.00` (> 7,5kg a 10kg/L)	`3808.69.90` ou `3105.10.00`
8	**NCM Faixa 4 (> 10kg/L)**	`ncm_faixa4_acima_10kg`	String(10)	Máscara `0000.00.00` (> 10kg/L ou granel)	`3808.69.90`, `3105.90.90`, `2520.10.19`
9	**NCM Granel / Produto Técnico**	`ncm_granel_tecnico`	String(10)	NCM industrial intermediária / pura	`3808.69.90`, `3808.93.19`, `3808.91.20`
10	**Regra de Embalagem TIPI**	`regra_tipi_embalagem`	Text	Fundamentação legal da TIPI	`Nota Subpos. 2 Cap. 38 (SC COSIT 98.042/22)`
11	**Sistema de Registro MAPA**	`sistema_registro_mapa`	String(150)	Norma regulatória do insumo no MAPA	`AGROFIT/MAPA — Produto Formulado (Lei nº 14.785/2023)`
12	**Enquadramento PIS/COFINS**	`enquadramento_pis_cofins`	String(150)	Beneficiado, Condicional ou Tributado	`Beneficiado (Lei 10.925/04): 0% até Mar/26 | 0,925% Real pós-Abr/26`

#	CAMPO NA PLANILHA	IDENTIFICADOR SNAKE_CASE	TIPO DE DADO	RESTRIÇÃO / VALIDAÇÃO	EXEMPLO REAL NA BASE
13	**CST Recomendado (Saída / Entrada)**	`cst_recomendado`	String(80)	CSTs oficiais na venda e na entrada	`Saída: CST 06 (c/ Ajuste M220/M620) | Entrada: CST 73`

3. Pipeline Transacional de Dupla Gravação (Dual-Sync Ingestion Pipeline)

3.1. O Desafio da Ingestão Concorrente

Quando uma nova integração aos portais do MAPA (AGROFIT, SIPEAGRO, Diário Oficial da União) ou uma tela de pré-cadastro do ERP envia um novo registro, o sistema não pode salvar um produto na Base Mestre sem antes certificar-se de que a molécula correspondente existe na Matriz de Princípios Ativos. Caso contrário, ocorreria uma violação de chave estrangeira ou o produto ficaria órfão de regras fiscais.

3.2. Fluxo Atômico do Algoritmo Dual-Sync

O motor de integração deve executar obrigatoriamente as seguintes quatro etapas sob uma única transação de banco de dados (ACID):

1. **Etapa 1 (Resolução ou Criação do Princípio Ativo):** O conector extrai o ingrediente ativo ou garantia analítica do payload. O sistema realiza busca na tabela `matriz_principios_ativos`:
   - *Se já existir:* recupera o `id` do princípio ativo existente e suas regras fiscais pré-validadas.
   - *Se NÃO existir (molécula ou garantia inédita):* aciona imediatamente o **Motor Químico-Aduaneiro**, que avalia a Nota 2 do Cap. 38, deduz as 4 faixas de NCM, determina a regra TIPI e o regime de PIS/COFINS, inserindo um **novo registro canônico** na `matriz_principios_ativos` e devolvendo o novo `id` gerado.
2. **Etapa 2 (Higienização e Normalização do Produto):** O conector extrai a marca comercial, limpa sufixos de embalagem (gerando a `marca_padronizada`), formata o número do registro MAPA (prefixo `Nº` ou código `TC`) e captura o titular do registro.
3. **Etapa 3 (Inserção Vinculada na Base Mestre):** O produto é inserido na tabela `cadastro_mestre_produtos`, preenchendo os 28 campos canônicos, associando a chave estrangeira `principio_ativo_id` ao registro resolvido na Etapa 1 e herdando as alíquotas e NCMs por faixa de embalagem.
4. **Etapa 4 (Confirmação Transacional - Commit):** A transação relacional é confirmada. Se qualquer validação falhar (ex.: NCM inválida ou erro no parsing do registro MAPA), é executado **Rollback total**, impedindo a criação de registros órfãos ou inconsistentes.

```
# Pseudocodigo Estruturado do Pipeline Dual-Sync
def sincronizar_registro_portal(raw_payload: dict, session: Session) -> ProdutoMaster:
    with session.begin(): # Garante atomicidade transacional (ACID)
        # 1. Normaliza string do principio ativo
        pa_nome = normalizar_texto_quimico(raw_payload["ingrediente_ativo"])
```

```
        # 2. Busca ou cria o principio ativo na Matriz (13 Campos)
        pa_record = session.query(MatrizPrincipioAtivo).filter_by(principio_ativo_chave=pa_nome).f
irst()
        if not pa_record:
            regras_fiscais = motor_quimico.classificar_molecula(pa_nome, raw_payload.get("class
e"))
            pa_record = MatrizPrincipioAtivo(
                principio_ativo_chave=pa_nome,
                macro_grupo_subgrupo_padrao=regras_fiscais.macro_subgrupo,
                nota_subpos_2="SIM" if regras_fiscais.eh_nota_2 else "NÃO",
                ncm_padrao_referencia=regras_fiscais.ncm_padrao,
                ncm_faixa1_ate_300g=regras_fiscais.ncm_f1,
                ncm_faixa2_300g_a_7_5kg=regras_fiscais.ncm_f2,
                ncm_faixa3_7_5kg_a_10kg=regras_fiscais.ncm_f3,
                ncm_faixa4_acima_10kg=regras_fiscais.ncm_f4,
                ncm_granel_tecnico=regras_fiscais.ncm_granel,
                regra_tipi_embalagem=regras_fiscais.regra_tipi,
                sistema_registro_mapa=regras_fiscais.sistema_mapa,
                enquadramento_pis_cofins=regras_fiscais.regime_pis,
                cst_recomendado=regras_fiscais.cst_str
            )
            session.add(pa_record)
            session.flush() # Gera pa_record.id imediatamente para a FK

        # 3. Insere o Produto na Base Mestre vinculado ao PA (28 Campos)
        produto_master = ProdutoMaster(
            principio_ativo_id=pa_record.id,
            nome_completo=raw_payload["nome_comercial"],
            marca_padronizada=sanitize_brand_name(raw_payload["nome_comercial"]),
            tipo_produto=inferir_tipo_produto(raw_payload),
            embalagem=raw_payload.get("embalagem", "PADRÃO COMERCIAL"),
            unidade=raw_payload.get("unidade", "L"),
            principio_ativo=pa_record.principio_ativo_chave,
            macro_grupo=extrair_macro(pa_record.macro_grupo_subgrupo_padrao),
            sub_grupo=extrair_sub(pa_record.macro_grupo_subgrupo_padrao),
            nota_subpos_2=pa_record.nota_subpos_2,
            ncm_padrao=pa_record.ncm_padrao_referencia,
            ncm_faixa1_ate_300g=pa_record.ncm_faixa1_ate_300g,
            ncm_faixa2_300g_a_7_5kg=pa_record.ncm_faixa2_300g_a_7_5kg,
            ncm_faixa3_7_5kg_a_10kg=pa_record.ncm_faixa3_7_5kg_a_10kg,
            ncm_faixa4_acima_10kg=pa_record.ncm_faixa4_acima_10kg,
            regra_tipi=pa_record.regra_tipi_embalagem,
            registro_mapa=fmt_mapa(raw_payload["registro_mapa"]),
            sistema_norma_mapa=pa_record.sistema_registro_mapa,
            titular_fabricante=raw_payload.get("titular", "NÃO INFORMADO"),
            origem_base=raw_payload.get("origem_base", "PORTAL_INTEGRACAO"),
            status_pis_cofins=pa_record.enquadramento_pis_cofins,
            aliquota_ate_mar2026=calcular_aliq_ate_mar26(pa_record),
            aliquota_pos_abr2026_real=calcular_aliq_real_pos_abr26(pa_record),
            aliquota_pos_abr2026_presumido=calcular_aliq_pres_pos_abr26(pa_record),
            cst_saida=extrair_cst_saida(pa_record.cst_recomendado),
            cst_entrada=extrair_cst_entrada(pa_record.cst_recomendado),
            ibs_cbs_2027=calcular_ibs_cbs(pa_record),
            url_oficial=raw_payload.get("url_oficial", "https://dados.agricultura.gov.br")
        )
        session.add(produto_master)
        # Ao sair do bloco 'with session.begin()', executa commit() automatico
    return produto_master
```

4. Implementação de Referência: Schemas Pydantic v2 e SQLAlchemy 2.0

4.1. Schemas de Validação Pydantic v2

Os schemas abaixo garantem a validação formal dos 13 campos da Matriz e expõem a relação com o produto mestre:

```
from pydantic import BaseModel, Field, field_validator
from typing import List, Optional
from enum import Enum

class MatrizPrincipioAtivoBase(BaseModel):
    principio_ativo_chave: str = Field(..., min_length=2, max_length=255, description="Nome quimic
o canônico")
    macro_grupo_subgrupo_padrao: str = Field(..., min_length=3, max_length=150)
    nota_subpos_2: str = Field(..., pattern="^(SIM|NÃO)$")
    ncm_padrao_referencia: str = Field(..., pattern=r"^\d{4}\.\d{2}\.\d{2}$")
    ncm_faixa1_ate_300g: str = Field(..., pattern=r"^\d{4}\.\d{2}\.\d{2}$")
    ncm_faixa2_300g_a_7_5kg: str = Field(..., pattern=r"^\d{4}\.\d{2}\.\d{2}$")
    ncm_faixa3_7_5kg_a_10kg: str = Field(..., pattern=r"^\d{4}\.\d{2}\.\d{2}$")
    ncm_faixa4_acima_10kg: str = Field(..., pattern=r"^\d{4}\.\d{2}\.\d{2}$")
    ncm_granel_tecnico: str = Field(..., pattern=r"^\d{4}\.\d{2}\.\d{2}$")
    regra_tipi_embalagem: str = Field(..., min_length=5)
    sistema_registro_mapa: str = Field(..., min_length=3, max_length=150)
    enquadramento_pis_cofins: str = Field(..., min_length=3, max_length=150)
    cst_recomendado: str = Field(..., min_length=3, max_length=80)

    @field_validator("principio_ativo_chave")
    def converter_para_maiusculo(cls, v: str) -> str:
        return v.strip().upper()

class MatrizPrincipioAtivoResponse(MatrizPrincipioAtivoBase):
    id: int
    total_produtos_vinculados: Optional[int] = 0

    class Config:
        from_attributes = True
```

4.2. Modelagem Relacional SQLAlchemy 2.0 (ORM)

O modelo relacional em Python mapeia a integridade referencial com cascata de atualização e restrição de exclusão de princípios ativos em uso:

```
from sqlalchemy import Column, Integer, String, Text, ForeignKey, Index
from sqlalchemy.orm import declarative_base, relationship

Base = declarative_base()

class MatrizPrincipioAtivoORM(Base):
    __tablename__ = "matriz_principios_ativos"

    id = Column(Integer, primary_key=True, autoincrement=True)
    principio_ativo_chave = Column(String(255), unique=True, nullable=False, index=True)
    macro_grupo_subgrupo_padrao = Column(String(150), nullable=False)
    nota_subpos_2 = Column(String(3), nullable=False)
    ncm_padrao_referencia = Column(String(10), nullable=False)
    ncm_faixa1_ate_300g = Column(String(10), nullable=False)
```

```
    ncm_faixa2_300g_a_7_5kg = Column(String(10), nullable=False)
    ncm_faixa3_7_5kg_a_10kg = Column(String(10), nullable=False)
    ncm_faixa4_acima_10kg = Column(String(10), nullable=False)
    ncm_granel_tecnico = Column(String(10), nullable=False)
    regra_tipi_embalagem = Column(Text, nullable=False)
    sistema_registro_mapa = Column(String(150), nullable=False)
    enquadramento_pis_cofins = Column(String(150), nullable=False)
    cst_recomendado = Column(String(80), nullable=False)

    # Relacionamento 1:N com Cadastro Mestre
    produtos_comerciais = relationship(
        "ProdutoMasterORM",
        back_populates="principio_ativo_rel",
        cascade="all, delete-orphan",
        passive_deletes=True
    )

class ProdutoMasterORM(Base):
    __tablename__ = "cadastro_mestre_produtos"

    id = Column(Integer, primary_key=True, autoincrement=True)
    principio_ativo_id = Column(
        Integer,
        ForeignKey("matriz_principios_ativos.id", ondelete="RESTRICT", onupdate="CASCADE"),
        nullable=False,
        index=True
    )
    nome_completo = Column(String(255), nullable=False)
    marca_padronizada = Column(String(150), nullable=False, index=True)
    tipo_produto = Column(String(80), nullable=False)
    embalagem = Column(String(100), nullable=False)
    unidade = Column(String(10), nullable=False)
    principio_ativo = Column(String(255), nullable=False)
    macro_grupo = Column(String(100), nullable=False)
    sub_grupo = Column(String(100), nullable=False)
    nota_subpos_2 = Column(String(3), nullable=False)
    ncm_padrao = Column(String(10), nullable=False, index=True)
    ncm_faixa1_ate_300g = Column(String(10), nullable=False)
    ncm_faixa2_300g_a_7_5kg = Column(String(10), nullable=False)
    ncm_faixa3_7_5kg_a_10kg = Column(String(10), nullable=False)
    ncm_faixa4_acima_10kg = Column(String(10), nullable=False)
    regra_tipi = Column(Text, nullable=False)
    registro_mapa = Column(String(50), nullable=False, index=True)
    sistema_norma_mapa = Column(String(150), nullable=False)
    titular_fabricante = Column(String(200), nullable=False)
    origem_base = Column(String(80), nullable=False)
    status_pis_cofins = Column(String(150), nullable=False)
    aliquota_ate_mar2026 = Column(String(50), nullable=False)
    aliquota_pos_abr2026_real = Column(String(50), nullable=False)
    aliquota_pos_abr2026_presumido = Column(String(50), nullable=False)
    cst_saida = Column(String(30), nullable=False)
    cst_entrada = Column(String(30), nullable=False)
    ibs_cbs_2027 = Column(String(150), nullable=False)
    url_oficial = Column(String(255), nullable=False)

    # Vínculo inverso
    principio_ativo_rel = relationship("MatrizPrincipioAtivoORM", back_populates="produtos_comerci
ais")
```

4.3. DDL SQL Completo com Foreign Key e Índices (PostgreSQL / SQLite)

O script SQL a seguir pode ser executado diretamente em ambientes PostgreSQL ou SQLite para criar a arquitetura relacional vinculada:

```
-- 1. TABELA DE REGRA FISCAL: MATRIZ DE PRINCIPIOS ATIVOS (13 COLUNAS)
CREATE TABLE IF NOT EXISTS matriz_principios_ativos (
    id SERIAL PRIMARY KEY,
    principio_ativo_chave VARCHAR(255) UNIQUE NOT NULL,
    macro_grupo_subgrupo_padrao VARCHAR(150) NOT NULL,
    nota_subpos_2 VARCHAR(3) NOT NULL CHECK (nota_subpos_2 IN ('SIM', 'NÃO')),
    ncm_padrao_referencia VARCHAR(10) NOT NULL,
    ncm_faixa1_ate_300g VARCHAR(10) NOT NULL,
    ncm_faixa2_300g_a_7_5kg VARCHAR(10) NOT NULL,
    ncm_faixa3_7_5kg_a_10kg VARCHAR(10) NOT NULL,
    ncm_faixa4_acima_10kg VARCHAR(10) NOT NULL,
    ncm_granel_tecnico VARCHAR(10) NOT NULL,
    regra_tipi_embalagem TEXT NOT NULL,
    sistema_registro_mapa VARCHAR(150) NOT NULL,
    enquadramento_pis_cofins VARCHAR(150) NOT NULL,
    cst_recomendado VARCHAR(80) NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_matriz_pa_chave ON matriz_principios_ativos(principio_ativo_chave);
CREATE INDEX idx_matriz_nota2 ON matriz_principios_ativos(nota_subpos_2);

-- 2. TABELA OPERACIONAL: CADASTRO MESTRE DE PRODUTOS VINCULADO (28 COLUNAS + FK)
CREATE TABLE IF NOT EXISTS cadastro_mestre_produtos (
    id SERIAL PRIMARY KEY,
    principio_ativo_id INTEGER NOT NULL REFERENCES matriz_principios_ativos(id) ON UPDATE CASCADE
ON DELETE RESTRICT,
    nome_completo VARCHAR(255) NOT NULL,
    marca_padronizada VARCHAR(150) NOT NULL,
    tipo_produto VARCHAR(80) NOT NULL,
    embalagem VARCHAR(100) NOT NULL,
    unidade VARCHAR(10) NOT NULL,
    principio_ativo VARCHAR(255) NOT NULL,
    macro_grupo VARCHAR(100) NOT NULL,
    sub_grupo VARCHAR(100) NOT NULL,
    nota_subpos_2 VARCHAR(3) NOT NULL CHECK (nota_subpos_2 IN ('SIM', 'NÃO')),
    ncm_padrao VARCHAR(10) NOT NULL,
    ncm_faixa1_ate_300g VARCHAR(10) NOT NULL,
    ncm_faixa2_300g_a_7_5kg VARCHAR(10) NOT NULL,
    ncm_faixa3_7_5kg_a_10kg VARCHAR(10) NOT NULL,
    ncm_faixa4_acima_10kg VARCHAR(10) NOT NULL,
    regra_tipi TEXT NOT NULL,
    registro_mapa VARCHAR(50) NOT NULL,
    sistema_norma_mapa VARCHAR(150) NOT NULL,
    titular_fabricante VARCHAR(200) NOT NULL,
    origem_base VARCHAR(80) NOT NULL,
    status_pis_cofins VARCHAR(150) NOT NULL,
    aliquota_ate_mar2026 VARCHAR(50) NOT NULL,
    aliquota_pos_abr2026_real VARCHAR(50) NOT NULL,
    aliquota_pos_abr2026_presumido VARCHAR(50) NOT NULL,
    cst_saida VARCHAR(30) NOT NULL,
    cst_entrada VARCHAR(30) NOT NULL,
    ibs_cbs_2027 VARCHAR(150) NOT NULL,
    url_oficial VARCHAR(255) NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE INDEX idx_cadmestre_pa_id ON cadastro_mestre_produtos(principio_ativo_id);
CREATE INDEX idx_cadmestre_regmapa ON cadastro_mestre_produtos(registro_mapa);
CREATE INDEX idx_cadmestre_marca_pad ON cadastro_mestre_produtos(marca_padronizada);
CREATE INDEX idx_cadmestre_ncm_padrao ON cadastro_mestre_produtos(ncm_padrao);
```

---

5. Especificação da API RESTful de Ingestão Integrada

5.1. Endpoint de Importação Atômica Dual

O endpoint abaixo é acionado na importação de novos cadastros (seja por upload de CSV de dados abertos do MAPA, leitura de feeds do Diário Oficial da União ou formulário de cadastro no portal):

```
# Router FastAPI de Ingestao Integrada
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from pydantic import BaseModel
from typing import Optional

router = APIRouter(prefix="/api/v1/integracoes", tags=["Ingestao Integrada MAPA"])

class IngestaoPortalInputDTO(BaseModel):
    nome_comercial_portal: str
    principio_ativo_declarado: str
    numero_registro_mapa: str
    classe_agronomica: Optional[str] = "Defensivos"
    titular_registro: Optional[str] = "Não informado"
    embalagem_comercial: Optional[str] = "GALÃO 20L"
    unidade_medida: Optional[str] = "L"
    origem_portal: str = "AGROFIT_IMPORT"
    url_publicacao_dou: Optional[str] = "https://dados.agricultura.gov.br"

class IngestaoDualResponseDTO(BaseModel):
    sucesso: bool
    mensagem: str
    principio_ativo_id: int
    principio_ativo_chave: str
    principio_ativo_novo_criado: bool
    produto_mestre_id: int
    produto_mestre_nome: str
    registro_mapa: str
    ncm_padrao: str
    cst_saida: str

@router.post(
    "/importar-novo-cadastro",
    response_model=IngestaoDualResponseDTO,
    status_code=status.HTTP_201_CREATED,
    summary="Importa registro oficial preenchendo atomicamente a Matriz de PAs e a Base Mestre"
)
async def importar_registro_portal(
    payload: IngestaoPortalInputDTO,
    db: Session = Depends(get_db_session)
):
    try:
        resultado = servico_ingestao.processar_dupla_gravacao(payload, db)
        return resultado
```

```
    except Exception as exc:
        db.rollback()
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=f"Erro no pipeline transacional de dupla gravação: {str(exc)}"
        )
```

5.2. Payload de Resposta da Ingestão Integrada

A resposta confirma o preenchimento simultâneo de ambas as bases e indica se o princípio ativo já era existente ou se foi criado como uma nova entrada na matriz:

```
{
  "sucesso": true,
  "mensagem": "Cadastro processado com sucesso em ambas as bases canônicas.",
  "principio_ativo_id": 42,
  "principio_ativo_chave": "GLIFOSATO 480 G/L SL",
  "principio_ativo_novo_criado": false,
  "produto_mestre_id": 1014,
  "produto_mestre_nome": "PILARSATO 1X20L",
  "registro_mapa": "Nº 2500",
  "ncm_padrao": "3808.93.24",
  "cst_saida": "CST 06"
}
```

---

6. Governança, CI/CD e Testes de Conformidade Relacional

6.1. Propagação Tributária em Cascata (Trigger ou Domain Event)

Quando a legislação ou ato declaratório alterar a tributação de um princípio ativo na `matriz_principios_ativos`, o sistema deve disparar a sincronização dos produtos dependentes:

```
-- Trigger PostgreSQL para Sincronizacao em Cascata
CREATE OR REPLACE FUNCTION fn_sync_produto_master_on_pa_update()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE cadastro_mestre_produtos
    SET
        nota_subpos_2 = NEW.nota_subpos_2,
        ncm_padrao = NEW.ncm_padrao_referencia,
        ncm_faixa1_ate_300g = NEW.ncm_faixa1_ate_300g,
        ncm_faixa2_300g_a_7_5kg = NEW.ncm_faixa2_300g_a_7_5kg,
        ncm_faixa3_7_5kg_a_10kg = NEW.ncm_faixa3_7_5kg_a_10kg,
        ncm_faixa4_acima_10kg = NEW.ncm_faixa4_acima_10kg,
        regra_tipi = NEW.regra_tipi_embalagem,
        status_pis_cofins = NEW.enquadramento_pis_cofins,
        atualizado_em = CURRENT_TIMESTAMP
    WHERE principio_ativo_id = NEW.id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_atualiza_produtos_em_cascata
AFTER UPDATE ON matriz_principios_ativos
```

```
FOR EACH ROW
EXECUTE FUNCTION fn_sync_produto_master_on_pa_update();
```

6.2. Casos de Teste Automatizados Mandatórios (Pytest)

- **Teste 1 (Dupla Gravação com Princípio Ativo Inédito):** Importar um produto com molécula nova (ex.: *Isocicloseram*). O teste deve verificar se: (a) um novo registro com 13 campos foi gerado na `matriz_principios_ativos`; (b) um novo registro com 28 campos foi gerado na `cadastro_mestre_produtos`; (c) a chave estrangeira `principio_ativo_id` aponta exatamente para o ID gerado.
- **Teste 2 (Reaproveitamento com Vínculo N:1):** Importar 3 marcas de Clomazona distintas. O teste deve confirmar que a `matriz_principios_ativos` possui apenas 1 registro único para Clomazona, e os 3 produtos compartilham o mesmo `principio_ativo_id`.
- **Teste 3 (Rollback em Falha Cadastral):** Simular falha de validação na NCM do produto mestre durante a ingestão. O teste deve comprovar que a transação fez rollback completo e nenhum princípio ativo órfão permaneceu gravado na matriz.
- **Teste 4 (Restrição de Deleção):** Tentar deletar um princípio ativo que possui produtos vinculados no cadastro mestre. O banco de dados deve levantar erro de integridade referencial (`FOREIGN KEY constraint failed / ON DELETE RESTRICT`).

6.3. Mapeamento no Banco de Dados dos Sistemas ERP

Para integrar essa arquitetura aos ERPs líderes de mercado:

- **TOTVS Protheus:** A Matriz de Princípios Ativos é mapeada na tabela genérica `SX5` ou tabela customizada `SZ1` (Princípios Ativos e Regras Tributárias). O cadastro de produtos (`SB1`) recebe o campo customizado `B1_XPAID` apontando para a `SZ1`, garantindo que o gatilho da TES Inteligente leia a NCM e a alíquota da tabela pai.
- **SAP S/4HANA:** A Matriz opera como *Custom Business Object* (CBO) no SAP BTP, enquanto a tabela `MARA` (Dados Gerais do Material) referencia o código do princípio ativo, alimentando automaticamente o módulo de determinação de impostos (Tax Determination Engine).